

Funções II: âmbito, composição e testes

Questão 1 · O mesmo nome em dois âmbitos

Regista as saídas de cada versão. Explica por que o valor exterior permanece inalterado.

Original: Dentro: 3; Fora: 10. Alterando a variável local: Dentro: 7; Fora: 10. Com o parâmetro e mostrar(20): Dentro: 20; Fora: 10.

```
x = 10
def mostrar(x):
    print('Dentro:', x)

mostrar(20)
print('Fora:', x)
```

O parâmetro é local à chamada. Nenhuma das atribuições ou chamadas apresentadas reatribui o x global.

Questão 2 · Compor resultados

Mostra os passos de cálculo das duas composições com 4. Por que inverter a ordem das funções pode alterar o resultado?

Com 3, `quadrado(dobro(3))` vale 36 e `dobro(quadrado(3))` vale 18. Com 4, os resultados são 64 e 32. Cálculos: `dobro(4) = 8`, `quadrado(8) = 64`; `quadrado(4) = 16`, `dobro(16) = 32`.

```
def dobro(n):
    return n * 2

def quadrado(n):
    return n * n

intermedio = dobro(4)
print(quadrado(intermedio))
intermedio = quadrado(4)
print(dobro(intermedio))
```

As operações não são intercambiáveis: elevar o dobro ao quadrado e dobrar o quadrado são cálculos diferentes.

Funções II: âmbito, composição e testes

Questão 3 · Isolar cálculos da entrada e saída

Que parte pode ser testada sem input? Explica a vantagem de manter a leitura fora da função de cálculo.

O exemplo com preço 100 e taxa 0.20 apresenta Total: 80.0. A chamada direta `preco_final(50, 0.10)` devolve 45.0. Se for impressa antes de `main()`, 45.0 aparece antes das leituras interativas.

`preco_final` pode ser testada com argumentos fixos, sem input. A função `main` coordena a interação. Esta separação permite reutilizar o cálculo e testar casos de forma automática.

Questão 4 · Testar com assert

Qual teste detetou a alteração incorreta? Explica por que testar apenas (0, 0) não seria suficiente.

O teste adicional pode ser `assert media(20, 20) == 20`. Ao remover os parênteses, o primeiro teste falha: $10 + 14 / 2$ dá 17.0, não 12. A execução interrompe-se nesse assert, antes da mensagem final.

O caso (0, 0) isolado passaria nas duas versões, porque ambas produzem zero. É necessário usar casos capazes de distinguir o cálculo correto do incorreto.

```
def media(a, b):  
    return (a + b) / 2  
  
assert media(10, 14) == 12  
assert media(0, 0) == 0  
assert media(20, 20) == 20  
print("Testes concluídos")
```

Funções II: âmbito, composição e testes

Questão 5 · Desafio final - uma solução possível

```
def nota_valida(n):  
    return 0 <= n <= 20  
  
def media(a, b):  
    return (a + b) / 2  
  
def classificar(n):  
    if n >= 10:  
        return 'Aprovado'  
    return 'Reprovado'  
  
def main():  
    a = float(input('Nota 1: '))  
    b = float(input('Nota 2: '))  
    if not (nota_valida(a) and nota_valida(b)):  
        print('Notas inválidas')  
        return  
    m = media(a, b)  
    print('Média:', m)  
    print(classificar(m))  
  
assert nota_valida(0)  
assert nota_valida(20)  
assert not nota_valida(-1)  
assert not nota_valida(21)  
assert media(8, 12) == 10  
assert classificar(10) == 'Aprovado'  
assert classificar(9) == 'Reprovado'  
main()
```

8 e 12: média 10.0, Aprovado. 8 e 10: média 9.0, Reprovado. -1 e 12: Notas inválidas, sem média. 20 e 20: média 20.0, Aprovado. Assume-se entrada numérica. O return antecipado em main termina o processamento de dados inválidos.